Optimization-Guided Adaptive Detector Operating-Point Control for Multi-Object Aerial Tracking

Ahmed Gouda Ismail* and Mohamed S. Mohamed†
Military Technical College, Cairo, Egypt

Tarek Ahmed Mahmoud‡
Egypt University of Informatics, Cairo, Egypt

Multi-object tracking in unmanned aerial vehicle video is difficult because object density, size, illumination, and blur change over time, while most tracking-by-detection systems run the detector with fixed settings. This paper proposes a training-free controller that adjusts the detector operating point online: the confidence threshold, the overlap threshold of non-maximum suppression, and the input resolution. Using only the current frame and past tracker output, it selects one of three operating points without modifying or retraining the detector or the tracker. On the VisDrone2019 dataset with ByteTrack, the controller raises the higher order tracking accuracy from 41.586 to 44.523, the multiple object tracking accuracy from 9.876 to 27.871, and the identity F1 score from 48.569 to 53.929, and it lowers identity switches from 1,341 to 1,016. With OATrack, the three scores rise from 46.770 to 48.095, from 28.165 to 34.483, and from 57.107 to 59.364, and identity switches fall from 773 to 604. Paired bootstrap analysis supports these improvements. On a second aerial benchmark, the controller raises the multiple object tracking accuracy by 17.601 and 3.267 points with ByteTrack and OATrack without retuning. It adds about 4.2 milliseconds per frame on a Tesla T4 graphics processor.

Nomenclature

a_i = area of tracked box i, pixels^2

$\mathcal{B}_{1:t-1}$ = boxes reported by the tracker for frames 1 to $t-1$

C_j = crowd cue at analysis step j

c_t = score threshold used at frame t

d_j = drop counter of the recovery probe

e_t = edge density of I_t

g_j = target state at analysis step j

*M.Sc. Student, Computer Engineering and Artificial Intelligence Department; corresponding author, a7medgouda1@gmail.com.
†Professor, Computer Engineering and Artificial Intelligence Department, mohamedms@mtc.edu.eg.
‡Associate Professor, Computer Engineering Department, tarek.mohamed@eui.edu.eg.

H_j = set of the last seven analysis steps

I_t = image of frame t

j = analysis step (frames $1, 11, 21, \ldots$)

N_j = number of tracked boxes in frame $t - 1$

P_j = peak object count of the recovery probe

q_j = controller state: LOW, MEDIUM, or HIGH

r_t = input resolution used at frame t, pixels

S_j = scene complexity index

T_j = tiny-object ratio

t = frame index

t_c = frame of the last state change

β_t = brightness of I_t

ℓ_j = scene label

η_t = intersection-over-union threshold of non-maximum suppression used at frame t

θ_t = detector operating point (c_t, η_t, r_t)

ν_t = sharpness of I_t

π = control rule

τ_m, τ_h = scene complexity index thresholds for MEDIUM and HIGH

I. Introduction

Unmanned aerial vehicles (UAVs) are widely used for traffic monitoring, surveillance, and search, and many of these tasks need multi-object tracking (MOT): every object must be detected in every frame and keep the same identity over time [1, 2]. Aerial video makes this hard. Objects are small, their number changes quickly, the camera moves, and illumination and blur change within one video [2–4]. Many of these applications also need results with low delay, so the processing cost matters as much as the accuracy.

Aerial MOT depends on detection quality, on association across frames, and on the available computation. These factors interact: missed detections break trajectories, false detections cause wrong associations and identity switches, and better detection usually costs more time. Balancing them is a central problem of aerial MOT systems.

Most MOT systems follow the tracking-by-detection design. A detector finds the objects in each frame, and a tracker links the detections over time [5, 6]. Much research improves the tracker, for example its motion model or its association rules, and much research improves the detector itself. Far less attention is given to how the detector is running. In practice the detector uses one confidence threshold, one non-maximum-suppression (NMS) threshold, and one input

resolution for the whole video. These three values form the detector operating point, and they decide which boxes reach the tracker.

A fixed operating point does not fit a scene that keeps changing. A setting that works in a sparse, clear scene can keep too much clutter in a dense scene. A setting that works in a dense scene can lose small or weak objects in another part of the same video. A large input helps small objects, but it costs the same time on easy and hard frames. The choice therefore moves false positives, missed objects, identity switches, and runtime together.

Adaptive inference methods change the detector configuration but usually need an extra learned model and target detection, not tracking [7, 8], and adaptive tracking methods change thresholds inside one specific tracker [9, 10]. What is missing is a simple, training-free way to adapt the detector operating point online for MOT, without changing the detector or the tracker.

This paper presents Adaptive Control for Multi-Object Tracking (AC-MOT), a training-free control layer that treats the detector operating point as a control variable of the tracking system rather than as a fixed setting. Every ten frames, AC-MOT reads inexpensive scene cues from the current image and from the boxes the tracker has already reported, estimates how crowded the scene is, and selects one of three operating points. The detector and the tracker are not retrained or modified; what is new is the joint, online control of the score threshold, the NMS threshold, and the input resolution for MOT, decided only from information that is available at run time.

The main contributions are:

1) AC-MOT, a training-free and causal control layer that treats the detector operating point as part of the MOT system and adapts it online, with the detector and the tracker unchanged (Sec. III).
2) An optimization-guided development path whose final thresholds and actions are selected by cross-validation on calibration sequences (Sec. IV).
3) An evaluation on VisDrone2019-MOT with ByteTrack and OATrack, using eight MOT metrics and paired bootstrap intervals, and a transfer to UAVDT without retuning (Sec. V).
4) A latency measurement showing that the control layer adds only a few milliseconds per frame on a Tesla T4 GPU (Sec. VI).

Section II describes the datasets and metrics and reviews related work, Sec. III specifies AC-MOT, Sec. IV explains how optimization experiments led to the frozen design, Sec. V gives the results, Sec. VI discusses them, and Sec. VII concludes the paper.

II. Background and Related Work

This section introduces the datasets and evaluation metrics used in this study, followed by a review of related work on MOT.

A. Dataset Description

Two public aerial video datasets are used: VisDrone2019-MOT [2] to develop and evaluate AC-MOT, and UAVDT [3] to test whether the same controller works on new data without changing its settings. Both contain UAV videos but differ in object categories, scenes, and recording conditions, and both are evaluated with ByteTrack and OATrack, with the detector and the trackers unchanged.

1. VisDrone2019-MOT

VisDrone2019-MOT is a widely used benchmark for MOT in UAV videos [2]. It contains video sequences recorded by various drone-mounted cameras in 14 cities in China, in urban and rural regions and in both sparse and crowded scenes. The VisDrone benchmark annotates ten object categories, such as pedestrians, cars, buses, and bicycles, with boxes that give the location and identity of each tracked object. Its MOT track uses 96 video clips: 56 for training (24,198 frames), 7 for validation (2,846 frames), 16 for challenge testing (6,322 frames), and 17 for development testing (6,635 frames). Although ten categories are annotated, the official MOT evaluation scores only five of them: pedestrian, car, van, bus, and truck [2].

Many objects are small, their number changes quickly in crowded traffic, the moving camera changes position and viewing angle, and lighting, background, and occlusion vary. These properties make the dataset suitable for studying detector settings: a larger input helps small objects, while the confidence and NMS thresholds change how many detections reach the tracker. Eight training clips serve as calibration data for the final controller (Sec. IV.D), and the seven validation clips serve as evaluation data (Sec. V).

2. UAVDT

UAVDT is another benchmark designed for object detection and MOT in UAV videos, with a particular focus on vehicles [3]. It consists of 100 video sequences selected from more than 10 hours of video recorded at urban locations such as squares, arterial streets, toll stations, highways, crossings, and T-junctions, at 30 frames per second (FPS) and a resolution of 1080×540 pixels. About 80,000 frames are annotated with 0.84 million boxes of vehicles in three categories: car, truck, and bus. The sequences are recorded under different weather conditions, flying altitudes, and camera viewing angles, which change object size, visibility, and the number of vehicles per frame. The test set contains 20 sequences for the MOT task [3], and these 20 sequences are used in this study (Sec. V).

B. MOT Evaluation Metrics

To evaluate the performance of the proposed AC-MOT framework, eight commonly used MOT metrics are considered: Higher Order Tracking Accuracy (HOTA), Multiple Object Tracking Accuracy (MOTA), Identity F1 Score (IDF1), identity switches (IDS), false positives (FP), false negatives (FN), precision, and recall. These metrics measure

different aspects of tracking performance, including detection accuracy, object identity consistency, and the ability to track objects across consecutive frames [11, 12]. They are defined below, starting from the errors counted in each frame and ending with the scores that combine them.

Detection errors. The first metrics count the errors made frame by frame. In each frame t, output boxes are matched one to one to ground-truth boxes with an intersection over union (IoU) of at least 0.5. Let GT_t be the number of ground-truth boxes, TP_t the number of matched pairs, FP_t the number of output boxes without a match (false positives), and FN_t the number of ground-truth boxes without a match (false negatives, or missed objects). Over all frames,

$$\mathrm{GT} = \sum_t \mathrm{GT}_t, \qquad \mathrm{TP} = \sum_t \mathrm{TP}_t, \tag{1}$$

$$\mathrm{FP} = \sum_t \mathrm{FP}_t, \qquad \mathrm{FN} = \sum_t \mathrm{FN}_t, \tag{2}$$

so that $\mathrm{TP} = \mathrm{GT} - \mathrm{FN}$. From these counts, precision is the fraction of output boxes that are correct, and recall is the fraction of ground-truth boxes that are found:

$$\mathrm{Precision} = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FP}}, \qquad \mathrm{Recall} = \frac{\mathrm{TP}}{\mathrm{GT}} \tag{3}$$

Identity switches. The counts above treat each frame separately, so they do not show whether an object keeps the same track over time. Identity switches measure this. Let $\phi_t(i)$ be the output track matched to ground-truth object i in frame t, and $\phi_{\mathrm{prev}}(i)$ the output track to which object i was last matched before t. An identity switch is counted whenever these two tracks differ:

$$\mathrm{IDS} = \sum_t \sum_i \mathbf{1}\left[\phi_t(i) \neq \phi_{\mathrm{prev}}(i)\right], \tag{4}$$

where the inner sum runs over the objects that are matched in frame t and were matched before.

MOTA and IDF1. These two scores summarize the errors above in a single number each. MOTA combines the three error types, and IDF1 measures how long each object keeps the correct identity:

$$\mathrm{MOTA} = 1 - \frac{\mathrm{FN} + \mathrm{FP} + \mathrm{IDS}}{\mathrm{GT}}, \tag{5}$$

$$\mathrm{IDF1} = \frac{2\,\mathrm{IDTP}}{2\,\mathrm{IDTP} + \mathrm{IDFP} + \mathrm{IDFN}} \tag{6}$$

For IDF1, each ground-truth track is first matched to at most one output track over the whole video. IDTP, IDFP, and IDFN are then the true-positive, false-positive, and false-negative boxes under this track-level matching. Because MOTA counts every false positive and every missed object, it reacts strongly to how many boxes the detector passes on. IDF1 depends more on whether each object keeps one identity.

HOTA. MOTA depends mostly on detection errors, and IDF1 mostly on identity errors. HOTA balances detection and association in one score [12]. At a localization threshold α, let TP_α, FN_α, and FP_α be the matched pairs, missed ground-truth boxes, and unmatched output boxes. For a matched pair c, $\mathrm{TPA}(c)$ is the set of matched pairs with the same ground-truth identity and the same output identity as c; $\mathrm{FNA}(c)$ is the set of boxes of the same ground-truth object that are matched to another output identity or not matched; and $\mathrm{FPA}(c)$ is the set of boxes of the same output track that are matched to another object or not matched. Then

$$\mathrm{DetA}_\alpha = \frac{|\mathrm{TP}_\alpha|}{|\mathrm{TP}_\alpha| + |\mathrm{FN}_\alpha| + |\mathrm{FP}_\alpha|}, \tag{7}$$

$$\mathcal{A}(c) = \frac{|\mathrm{TPA}(c)|}{|\mathrm{TPA}(c)| + |\mathrm{FNA}(c)| + |\mathrm{FPA}(c)|}, \tag{8}$$

$$\mathrm{AssA}_\alpha = \frac{1}{|\mathrm{TP}_\alpha|} \sum_{c \in \mathrm{TP}_\alpha} \mathcal{A}(c), \tag{9}$$

$$\mathrm{HOTA} = \frac{1}{19} \sum_{\alpha} \sqrt{\mathrm{DetA}_\alpha \, \mathrm{AssA}_\alpha}, \tag{10}$$

where α runs over $0.05, 0.10, \ldots, 0.95$.

Statistical comparison. To judge whether the difference between two systems is reliable, paired bootstrap resampling is applied at the sequence level [13], which keeps the dependence within each sequence. Each resample draws the evaluation sequences with replacement; the same draw is used for both systems, the pooled metrics are recomputed, and their difference is recorded. The 95% interval is the 2.5–97.5 percentile range.

Units and direction. HOTA, MOTA, IDF1, precision, and recall are given in percent; IDS, FP, and FN are counts; speed is given in FPS, and runtime in milliseconds (ms) per frame. Higher values are better for HOTA, MOTA, IDF1, precision, recall, and FPS, and lower values are better for IDS, FP, FN, and runtime.

C. MOT Literature Survey

Four lines of work relate to AC-MOT: tracking-by-detection methods, which provide the detector and tracker that AC-MOT controls; adaptive threshold methods, which change settings inside a tracker; adaptive inference methods, which change how a detector is running; and the optimization and evaluation tools used to develop and test AC-MOT.

1. Tracking-by-Detection Methods in Aerial MOT

In tracking-by-detection, objects are detected in each frame and associated across frames; most methods improve motion prediction or association for a given detector.

SORT built a compact online pipeline from motion prediction and assignment [5], and StrongSORT adds appearance features to reduce identity fragmentation [14]. ByteTrack first associates high-confidence detections and then uses low-confidence boxes to recover objects that would otherwise be lost [6]. TrackFormer performs detection and

association with transformer queries [15]. Standard evaluation follows the MOTChallenge benchmarks [11].

Aerial video adds small targets, changing object counts, camera motion, and clutter [2, 3]. UAVMOT addresses the motion of the UAV camera inside the tracker [16]. The aerospace literature has studied the same airborne perception problem, from tracking several ground targets with a multirotor UAV [17] and compact descriptors for moving-object tracking [18] to onboard deep detection [19, 20]. LSO-YOLO designs a lightweight real-time detector for small UAV objects [21]. OATrack [22] is a recent UAV small-object tracker published in 2026. It weights association by detection quality and uses a YOLO detection cache. We use OATrack as a second host and keep it unchanged. These works improve the tracker or the detector itself. AC-MOT asks a different question: which detector setting should be used for the current scene.

Detector and trackers used in this study. The experiments use the YOLO11m detector [23], a one-stage detector of the Ultralytics YOLO family, in a public version trained on the VisDrone2019-DET dataset [2]. Two of the trackers above serve as hosts. ByteTrack [6] is a widely used tracker whose second association step uses low-score boxes. OATrack [22] is a recent tracker for small UAV objects that weights association by detection quality. Both hosts are used without modification.

2. Adaptive Threshold Selection in MOT

Fixed tracker thresholds decide which detections are accepted and how they are associated, but their reliability changes with object density, occlusion, and detection confidence, so several methods adapt them.

Adaptive ByteTrack adjusts the confidence threshold of ByteTrack [9] to variations in detection confidence, and AdapTrack [10] adapts the matching thresholds of its data association. Both adapt inside the tracker and are designed around one tracking algorithm. AC-MOT instead adapts the detector operating point: the confidence threshold decides which weak detections are kept, the NMS IoU threshold controls how overlapping boxes are suppressed, and the input resolution sets both the visibility of small objects and the cost of detection. It works as an external controller, so the tracker keeps its own association and identity management, and the same controller serves ByteTrack and OATrack without modifying or retraining either tracker.

3. Adaptive Inference for Object Detection

Dynamic neural networks change their structure or parameters for each input [24], and resolution-adaptive networks spend less computation on easy inputs [25]. AdaScale selects the input scale of each video frame with a scale regressor trained on detector features [7]. ApproxDet switches detection configurations at run time with accuracy and latency models trained offline [8]. Chameleon re-profiles the configuration of video analytics pipelines over time [26]. These methods trade detection accuracy against cost, and they select a configuration with additional trained models or periodic profiling. They are designed for detection or video analytics, not for MOT, where a detection error can also break an

association or switch an identity.

AC-MOT adjusts the three detector parameters together for better tracking rather than better detection, needs no learned model, and is evaluated with tracking metrics on two trackers. Table 1 summarizes the differences from the closest adaptive methods.

Table 1 Adaptive methods closest to AC-MOT (BT: ByteTrack).

Method	Adapted	Where	Task
AdaScale [7]	input scale	detector	video detection
ApproxDet [8]	detection configuration	detector	video detection
Chameleon [26]	pipeline configuration	analytics pipeline	video analytics
Adapt. BT [9]	confidence threshold	inside BT	MOT
AdapTrack [10]	matching thresholds	inside the tracker	MOT
AC-MOT	confidence, NMS, resolution	detector; tracker unchanged	aerial MOT

4. Optimization and Evaluation for MOT

Detector and tracker parameters affect accuracy, identity consistency, and speed in conflicting ways, so they need systematic search and an evaluation that covers both detection and identity, with a test of reliability.

Optuna provides define-by-run search with the Tree-structured Parzen Estimator (TPE) sampler [27], which was used throughout the development of AC-MOT. The metrics and the paired bootstrap of Sec. II.B are used to compare AC-MOT with its baselines.

In summary, existing work adapts either the tracker's own thresholds or the detector's configuration for detection accuracy and cost. A training-free method that adapts the detector operating point online for MOT, with the detector and the tracker left unchanged, is missing. AC-MOT is designed to fill this gap.

III. Proposed AC-MOT Framework

AC-MOT is a small, rule-based controller that runs next to the detector. On frame 1 and then on every tenth frame, it measures cheap cues of the current image and of the boxes tracked in the previous frame (Sec. III.B), averages the recent crowd level into a scene complexity index (SCI) (Sec. III.C), chooses a target state (LOW, MEDIUM, or HIGH), adjusts it with a recovery probe and hysteresis and dwell rules (Sec. III.D), and maps the state to one of three fixed detector operating points (Sec. III.E). Section III.F explains how each value is chosen, and the Nomenclature lists the symbols.

A. Overview and operating point

Figure 1 shows where AC-MOT sits in the pipeline: the controller chooses the detector settings before the detector runs, and the detector and the tracker then run as usual. The detector settings at frame t are called the operating point

$$\theta_t = (c_t, \eta_t, r_t), \tag{11}$$

which has three parts:

- r_t is the input resolution, the long side of the resized image in pixels. A larger input shows small objects better but takes more time.
- η_t is the NMS IoU threshold. When two boxes overlap with an IoU above η_t, the box with the lower score is removed. A lower η_t removes more overlapping boxes.
- c_t is the score threshold. Boxes with a detector score below c_t are removed before they reach the tracker. A higher c_t passes fewer but more reliable boxes.

The detector weights and the tracker are frozen; AC-MOT changes only θ_t.

The controller uses only information that is available at run time:

$$\theta_t = \pi\left(I_t, \mathcal{B}_{1:t-1}\right), \tag{12}$$

where π is the control rule described in this section, I_t is the image of the current frame, and $\mathcal{B}_{1:t-1}$ are the boxes that the tracker reported for the earlier frames 1 to $t-1$. The tracker output of frame t is used only from frame $t+1$ on. No ground truth, no future frame, and no detector score enters the decision.

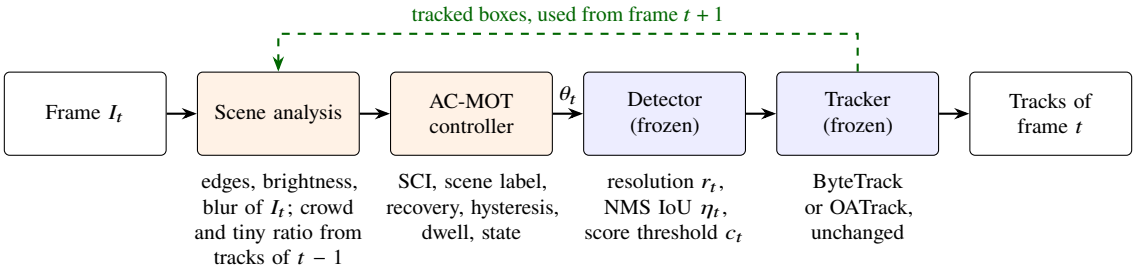


Fig. 1 AC-MOT in a tracking-by-detection pipeline. Scene analysis and the controller choose the operating point $\theta_t = (c_t, \eta_t, r_t)$ before the detector runs. The frozen detector and tracker are not modified. Tracked boxes of frame t feed the scene analysis of later frames (dashed arrow).

B. Scene cues

The controller needs a cheap description of the scene. Scene analysis therefore does not run on every frame. It runs on frame 1 and then once every 10 frames ($t = 1$ or $t \bmod 10 = 1$, that is, frames 1, 11, 21, ...). Each of these frames is

an analysis step j. Between two analysis steps the controller keeps its last decision. Two kinds of cues are computed at each step: image cues, which describe how the current frame looks, and box cues, which describe the objects that the tracker found in the previous frame.

Image cues of the current frame. The frame is reduced to a quarter of its width and height and converted to gray levels, so that the cues are cheap to compute. Three cues are then measured:

- the edge density e_t, the fraction of pixels marked as edges by the Canny detector (lower and upper thresholds 50 and 120); many edges indicate a cluttered scene;
- the brightness β_t, the mean gray level from 0 (black) to 255 (white); a low value indicates a dark or night scene;
- the sharpness ν_t, the variance of the Laplacian of the gray image; a low value indicates a blurred frame.

Box cues of the previous frame. Let N_j be the number of boxes that the tracker reported for frame $t-1$, and let a_i be the area of box i in pixels of the original image. The crowd cue and the tiny-object ratio are

$$C_j = \min\left(\frac{N_j}{30}, 1\right), \tag{13}$$

$$T_j = \frac{1}{N_j} \sum_{i=1}^{N_j} \mathbf{1}\left[a_i < 32^2\right] \tag{14}$$

C_j is the number of tracked objects divided by 30 and capped at 1, so every frame with 30 or more objects has $C_j = 1$. T_j is the fraction of tracked boxes smaller than 32×32 pixels, which is the size limit of small objects in the COCO benchmark [28]. The indicator $\mathbf{1}[\cdot]$ is 1 when its condition holds and 0 otherwise, and $T_j = 0$ when $N_j = 0$.

C. SCI and scene label

The SCI measures how crowded the recent frames are. A single frame is a noisy measurement, because the tracker may miss or add a few objects in any frame, so the SCI averages over time. It is the mean crowd cue of the last seven analysis steps:

$$S_j = \frac{1}{|H_j|} \sum_{k \in H_j} C_k, \tag{15}$$

where H_j holds the steps $j-6, \ldots, j$ (fewer at the start of a video) and $|H_j|$ is their number. With a stride of 10 frames, the seven steps cover about the last 60 frames, so a single unusual frame changes S_j only a little. Because $C_j \le 1$, S_j lies between 0 and 1.

The SCI uses the crowd cue alone, selected by the cue audit of Sec. IV.D. The other four cues are still used as checks

on the state, mainly through the scene label

$$\ell_j = \begin{cases} \text{night} & \text{if } \beta_t < 80, \\ \text{blur} & \text{else if } \nu_t < 180, \\ \text{tiny} & \text{else if } T_j > 0.50, \\ \text{crowded} & \text{else if } C_j > 0.65 \text{ or } e_t > 0.13, \\ \text{clear} & \text{otherwise.} \end{cases} \tag{16}$$

The conditions are checked from top to bottom, and the first one that holds gives the label. For example, $T_j > 0.50$ means that more than half of the tracked boxes are tiny, and $C_j > 0.65$ means that at least 20 objects are tracked.

Only the crowd cue enters the SCI, and it also enters the crowded label. The tiny-object ratio T_j is used by the state rules of Sec. III.D and gives the tiny label, and the edge density e_t is used only through the crowded label. The brightness β_t and the sharpness ν_t act only through the night and blur labels, which do not lead to MEDIUM or HIGH by themselves: because they are checked first, a dark or blurred frame cannot receive the crowded or tiny label, while the SCI conditions and the tiny-object guard of Eq. (17) still apply.

D. Three-state controller

The controller is always in one of three states: LOW, MEDIUM, or HIGH. The names describe the scene: LOW for a sparse scene, HIGH for a crowded scene or a scene of mostly tiny objects. They do not rank the input size: in the final action map the HIGH state uses the smallest input (Table 2). The state is updated only at analysis steps, in four steps (Algorithm 1). Step 1 proposes a target state from the cues. Steps 2–4 adjust this target so that the controller reacts to a sudden drop in the object count but does not switch back and forth.

Step 1: target state. The target is the state that the current cues ask for. It is given by the SCI, the tiny-object ratio, and the scene label:

$$g_j = \begin{cases} \text{HIGH} & \text{if } S_j > \tau_h \text{ or } T_j > 0.50, \\ \text{MEDIUM} & \text{else if } S_j > \tau_m \text{ or} \\ & \quad \ell_j \in \{\text{crowded}, \text{tiny}\}, \\ \text{LOW} & \text{otherwise,} \end{cases} \tag{17}$$

where $\tau_m = 0.297$ and $\tau_h = 0.511$ are the two SCI thresholds (Table 3). Since S_j is the recent average number of tracked objects divided by 30, the SCI conditions give MEDIUM above about 9 objects ($30\tau_m = 8.9$) and HIGH above about 15 objects ($30\tau_h = 15.3$).

Step 2: recovery probe. The SCI averages seven steps, so it reacts slowly when the tracker suddenly reports far

fewer objects than a few steps earlier. The recovery probe responds to such a collapse of the object count. Instead of letting the falling count move the controller toward a lower state, it sets the target to HIGH for a short time. To detect a collapse, it keeps a slowly decaying peak P_j of the object count and a drop counter d_j:

$$d_j = \begin{cases} d_{j-1} + 1 & \text{if } P_{j-1} \geq 5 \text{ and} \\ & \quad N_j < 0.40\, P_{j-1}, \\ 0 & \text{otherwise,} \end{cases} \tag{18}$$

$$P_j = \max\left(N_j, \lfloor 0.8\, P_{j-1} \rfloor\right), \tag{19}$$

with $P_0 = d_0 = 0$. A collapse is detected when at least 5 objects have been seen before and fewer than 40% of them remain. The peak loses 20% at each step, so an old peak is forgotten after a few steps. While $1 \leq d_j \leq 3$, the target g_j is set to HIGH, so the probe lasts at most three analysis steps (30 frames); the dwell rule of Step 4 still applies.

Step 3: hysteresis. Without a margin, a scene whose SCI stays close to a threshold would make the controller switch at almost every step. Hysteresis prevents this: a move to a lower state is held while the scene is still close to difficult:

$$g_j \leftarrow \begin{cases} \text{HIGH} & \text{if } q_{j-1} = \text{HIGH and} \\ & \quad (S_j > 0.50 \text{ or } T_j > 0.40), \\ \text{MEDIUM} & \text{if } q_{j-1} = \text{MEDIUM}, \\ & \quad g_j = \text{LOW and } S_j > 0.25, \\ g_j & \text{otherwise.} \end{cases} \tag{20}$$

The hold values lie just below the values that lead into each state ($0.50 < \tau_h$, 0.40 below the tiny guard of 0.50, and $0.25 < \tau_m$). Upward moves are not held.

Step 4: dwell. The dwell rule sets a minimum time between two changes. After a change, the state is kept for at least 30 frames, that is, three analysis steps:

$$q_j = \begin{cases} g_j & \text{if } t - t_c \geq 30, \\ q_{j-1} & \text{otherwise,} \end{cases} \tag{21}$$

where t_c is the frame of the last state change. The controller starts in LOW with $t_c = 1$, so the first change is possible at frame 31. Whenever $q_j \neq q_{j-1}$, t_c is set to t.

Why changes are kept rare. Each change of state changes the boxes that reach the tracker, so the temporal rules let the state follow lasting changes of the scene, not single frames: the seven-step window smooths the SCI over about 60

frames, the holds add a margin below each threshold, and the dwell rule allows at most one change every 30 frames.

Example. Consider a clear, sharp video in which the tracker reports 20 objects of normal size in every frame ($C_j = 0.67$), and only 10 objects from frame 71 on. The controller starts in LOW. The first step sees no earlier boxes ($C_j = 0$), so S_j grows as the window fills: 0.33 at frame 11, 0.44 at frame 21, and 0.50 at frame 31. The target is MEDIUM from frame 11, but the dwell rule keeps LOW until frame 31, where the state becomes MEDIUM. From frame 41, S_j exceeds τ_h and the target is HIGH; the dwell rule delays this change to frame 61. When the scene thins, S_j falls slowly because the window still holds the earlier steps, so the state stays HIGH until frame 101. At frame 111, $S_j = 0.48$ is below both τ_h and the hold value of 0.50, and the state returns to MEDIUM. It then stays in MEDIUM, because 10 tracked objects keep S_j at about 0.33, above τ_m.

The state q_j then selects the operating point from the action map, and the detector uses it until the next analysis step.

Algorithm 1 AC-MOT controller for one video

$j \leftarrow 0$; $q_0 \leftarrow$ LOW; $P_0 \leftarrow 0$; $d_0 \leftarrow 0$; $t_c \leftarrow 1$
for each frame $t = 1, 2, \ldots$ **do**
  if $t = 1$ **or** t mod 10 = 1 **then**
    $j \leftarrow j + 1$
    $e_t, \beta_t, \nu_t \leftarrow$ image cues of I_t
    $N_j, C_j, T_j \leftarrow$ box cues of frame $t - 1$ (Eqs. 13–14)
    $S_j \leftarrow$ mean of C over H_j, the last 7 steps (Eq. 15)
    $\ell_j \leftarrow$ label (Eq. 16); $g_j \leftarrow$ target (Eq. 17)
    update d_j and P_j (Eqs. 18–19)
    if $1 \le d_j \le 3$ **then** $g_j \leftarrow$ HIGH
    end if
    apply the holds to g_j (Eq. 20)
    if $t - t_c \ge 30$ **and** $g_j \ne q_{j-1}$ **then**
      $q_j \leftarrow g_j$; $t_c \leftarrow t$
    else
      $q_j \leftarrow q_{j-1}$
    end if
  end if
  $(r_t, \eta_t, c_t) \leftarrow$ action of state q_j (Table 2)
  detect with r_t and η_t; drop boxes with score $< c_t$
  update the tracker; store its boxes of frame t
end for

E. Action map

Table 2 gives the operating point of each state. The actions are not set by hand: they are selected together with τ_m and τ_h by the joint search on the calibration sequences, whose objective is the change in HOTA, a metric that balances detection and association (Sec. III.F).

In LOW, the sparse state, the detector uses the largest input, loose NMS, and a low score threshold, so it passes more weak boxes. In MEDIUM and HIGH it passes only boxes with a score of at least 0.40. The HIGH state, used for crowded scenes and for scenes of mostly tiny objects, thus has the smallest input. This may look surprising, since a

larger input shows small objects better, but the search optimizes tracking quality, not the detection of every small object: a score threshold of 0.40 gives the tracker fewer but more reliable boxes (Sec. IV.D), and the smaller input costs less detector time. Section V shows that the main effect of AC-MOT is a more selective detection stream: fewer FP and IDS, with similar or slightly lower recall. This action is the result of the search, not a general rule that crowded scenes need smaller inputs.

The score threshold c_t is applied on the detector side before the tracker. It is separate from any confidence gate inside the tracker, such as the fixed tracker-side gate of OATrack.

Table 2 Action map. State names describe scene difficulty, not a resolution rank. Selected by the joint search on the calibration sequences.

State	Resolution r	NMS IoU η	Score threshold c
LOW	1536	0.70	0.10
MEDIUM	1280	0.45	0.40
HIGH	1088	0.60	0.40

F. Selection of Parameter Values of the AC-MOT Controller

All values of Tables 2 and 3 are fixed before the final evaluation and are not changed after it. The last column of Table 3 gives the source of each value. There are four sources.

Joint search on the calibration sequences. The SCI thresholds τ_m and τ_h and the action map are selected together by an Optuna TPE search with 60 trials [27]; the search ranges are given in Sec. IV.D. The calibration sequences are eight sequences of the VisDrone2019-MOT training split. They serve as the validation (tuning) data of the final stage and are disjoint from the evaluation sequences. Each trial is scored by four-fold cross-validation, with two sequences held out in each fold, as the mean change in HOTA against the baseline. The candidate values come from existing settings. The input sizes (1088, 1280, and 1536 pixels) and NMS values (0.45, 0.60, and 0.70) are the operating points for which detector outputs are cached; 1536 pixels with NMS 0.70 is the baseline point, and 0.70 is the default NMS value of the Ultralytics detector. The three score thresholds are the low and high thresholds of ByteTrack (0.10 and 0.25) and the confidence gate of OATrack (0.40). The score-threshold sweep and the fold results are given in Sec. IV.D.

Cue audit on the calibration sequences. The choice of the crowd cue as the only SCI input comes from testing all 31 non-empty subsets of the five cues with the same cross-validation (Sec. IV.D).

Temporal sweep on the validation split. The analysis stride and the SCI window are selected in the first development stage, with the YOLOv8n pipeline, on the VisDrone2019-MOT validation sequences. The sweep tests 25 combinations: windows of 1, 3, 5, 7, and 9 analysis steps and strides of 1, 5, 10, 15, and 20 frames. Five of the 25 settings meet both constraints of that stage, at least 25 FPS and at most 271 IDS (the IDS of the original controller on the same data). Among them, a window of 7 and a stride of 10 give the highest MOTA, 18.165 against 18.044 for the next setting.

Operating-point sweeps on the same data cover input resolutions from 512 to 960 pixels, confidence thresholds from 0.05 to 0.50, and NMS thresholds from 0.30 to 0.80, and define the supported values of the first optimized profiles (Sec. IV).

Fixed design values. The tiny-object area follows the COCO definition of small objects [28]. The remaining values (the crowd normalizer, the Canny thresholds, the label thresholds, the recovery, hold, and dwell values, and the initial state) come from the original AC-MOT controller and are kept unchanged in every search of the final stage, so the searched values are matched to them. For example, the crowd normalizer of 30 only sets the scale of S_j: τ_m and τ_h are searched on this scale and correspond to about 9 and 15 tracked objects.

Table 3 Values of the AC-MOT controller and how each value is chosen. Calibration sequences: eight VisDrone2019-MOT training sequences used as tuning data, disjoint from the evaluation sequences. CV: four-fold cross-validation.

Symbol	Meaning	Value	How the value is chosen
τ_m	SCI threshold for MEDIUM, Eq. (17)	0.297	joint TPE search on the calibration sequences, 60 trials, CV
τ_h	SCI threshold for HIGH, Eq. (17)	0.511	same joint search
(r, η, c)	operating point of each state	Table 2	same joint search over $\{1088, 1280, 1536\} \times \{0.45, 0.60, 0.70\} \times \{0.10, 0.25, 0.40\}$
–	SCI input cue	crowd only	cue audit of all 31 cue subsets on the calibration sequences, CV
–	analysis stride	10 frames	temporal sweep of 25 settings on the validation split; highest MOTA with $\geq$25 FPS and $\leq$271 IDS
$\lvert H_j \rvert$	SCI window	7 steps	same temporal sweep
–	tiny-object area	32×32 pixels	COCO small-object limit [28]
–	crowd normalizer, Eq. (13)	30 boxes	original controller; sets the scale on which τ_m, τ_h are searched
–	Canny thresholds (lower, upper)	50, 120	original controller; fixed in all final-stage searches
–	night and blur labels, Eq. (16)	$\beta_t < 80$, $\nu_t < 180$	original controller; fixed in all final-stage searches
–	crowded label, Eq. (16)	$C_j > 0.65$ or $e_t > 0.13$	original controller; fixed in all final-stage searches
–	tiny label and HIGH guard, Eqs. (16), (17)	$T_j > 0.50$	original controller; fixed in all final-stage searches
–	recovery: minimum peak, drop ratio, peak decay, steps	5, 0.40, 0.80, 3	original controller; fixed in all final-stage searches
–	holds in HIGH (S_j, T_j) and MEDIUM (S_j), Eq. (20)	(0.50, 0.40), 0.25	original controller; fixed in all final-stage searches
–	dwell, Eq. (21)	30 frames	original controller; fixed in all final-stage searches
–	initial state	LOW	original controller; fixed in all final-stage searches

IV. Development of the Final AC-MOT Controller

The controller of Sec. III is the result of four development stages. The quality-oriented profile (Sec. IV.A) asked whether a scene index that steers the detector settings can beat a fixed default; the identity-aware profile (Sec. IV.B) made identity stability and speed explicit objectives; the transfer studies (Sec. IV.C) tested other data and pipelines; and the modern stage (Sec. IV.D) built the frozen controller for a modern detector with separate calibration data. The first three stages are historical: they used a weighted five-cue index with interpolated confidence and NMS values and three resolution levels (the SparseTrack study used a single edge-driven NMS rule), other detectors, and other evaluation

protocols. Their numbers are reported in their own tables, each against its own baseline, and are not pooled with the modern results.

A. Quality-oriented profile

Baseline. The reference pipeline is the pretrained YOLOv8n detector [29] followed by ByteTrack [6], both at their default settings: confidence threshold 0.25, NMS IoU threshold 0.45, input size 640 pixels, and the default ByteTrack parameters listed in Sec. V. It runs with one fixed operating point for the whole video.

Method. The first optimization-derived profile kept the same detector, used a tuned ByteTrack profile, and searched only on the VisDrone2019-MOT validation split. Each of five cues was normalized with validation statistics, and the index was their weighted sum

$$S_j = \sum_{k \in \{c,t,e,n,b\}} w_k \, x_{k,j}, \tag{22}$$

where k runs over the cues (c: crowd count, t: fraction of earlier boxes below 32×32 pixels, e: edge density, n: low brightness, b: low Laplacian variance), $x_{k,j}$ is the normalized value of cue k at analysis step j, and w_k is its weight, with $w_k \geq 0$ and $\sum_k w_k = 1$. The index was averaged over seven analyses taken every ten frames. Confidence and NMS were interpolated linearly between the values at $S = 0$ and $S = 1$, and two sorted thresholds selected one of three resolutions (512, 928, 960 pixels).

An Optuna TPE study [27] jointly searched the five weights, the confidence and NMS end points, and the two thresholds. The resolution levels, the supported confidence values $\{0.25, \ldots, 0.45\}$, the supported NMS values from 0.30 to 0.70, and the temporal settings were fixed beforehand by validation sweeps (Sec. III.F). The selection rule kept trials with at least 25 FPS and at most 271 IDS (the IDS of the original controller on the same validation data) and then took the highest MOTA, with lower IDS, HOTA, IDF1, and FPS as tie-breaks. The frozen profile (archived as Trial 24) has $w_c = 0.1294928$, $w_t = 0.2217477$, $w_e = 0.4337134$, $w_n = 0.0535577$, $w_b = 0.1614885$, confidence end points $(0.30, 0.40)$, NMS end points $(0.35, 0.35)$, and thresholds $(0.1353494, 0.2872868)$. On validation it reached MOTA 23.038, HOTA 36.110, IDF1 40.758, and IDS 270 at 37.169 FPS.

Comparison. Table 4 compares the profile with the baseline on the historical VisDrone test-dev split, with paired bootstrap intervals (5000 resamples, seed 42).

What this stage showed. A scene index that steers the detector settings improved tracking quality over the fixed default point, with HOTA, MOTA, and IDF1 intervals above zero. The IDS interval, $[-114, 219]$, includes zero; the next stage made identity stability an explicit objective. The final controller keeps two parts of this stage: the stride of 10 frames and the window of seven steps from its temporal sweep (Sec. III.F), and the five kinds of scene cues, which were audited again in the modern stage.

Table 4 Quality-oriented profile versus its baseline (YOLOv8n with ByteTrack at default settings) on VisDrone test-dev, historical protocol. Gain: paired bootstrap difference with 95% interval; for IDS it is the reduction.

Metric	Baseline	Quality	Gain [95% CI]
HOTA	28.430	33.835	+5.4051 [4.1273, 6.9022]
MOTA	19.729	26.948	+7.2195 [5.4362, 9.2934]
IDF1	32.724	41.546	+8.8220 [6.9005, 11.0537]
IDS	1235	1184	51 [−114, 219]
FPS	36.528	38.985	–

B. Identity-aware Pareto profile

Baseline. The baseline is the same YOLOv8n and ByteTrack pipeline at default settings (Sec. IV.A).

Method. The second study kept the same cues, operating levels, temporal settings, and validation data, but changed the question. It maximized MOTA and minimized IDS subject only to FPS ≥ 25, without the IDS gate. A seeded TPE sampler ran 49 completed trials. A trial dominated another when it had no lower MOTA and no higher IDS, with at least one strict improvement, and the feasible Pareto front was built from these relations. The balanced candidate maximized an equal-weight score of normalized MOTA and normalized IDS reduction, with HOTA, IDF1, FPS, and trial index as tie-breaks.

The selected profile (archived as Trial 22) has weights $w_c = 0.1646453$, $w_t = 0.1746265$, $w_e = 0.5076113$, $w_n = 0.1206956$, $w_b = 0.0324213$, confidence end points $(0.40, 0.40)$, NMS end points $(0.60, 0.35)$, and thresholds $(0.2927136, 0.6661672)$. On validation it reached MOTA 19.330, HOTA 31.651, IDF1 34.226, and IDS 168 at 51.406 FPS.

Comparison. Table 5 compares the profile with the baseline on test-dev. It removes 316 IDS and raises throughput from 36.528 to 46.024 FPS, while HOTA, MOTA, and IDF1 also rise. Against the quality-oriented profile it has 265 fewer IDS $[82, 502]$ and -2.617 HOTA $[-4.004, -1.694]$. The two profiles therefore answer different questions: one favors overall quality and the other favors identity stability and speed.

What this stage showed. The detector operating point controls a trade-off between quality, identity stability, and speed, and the objective of the search decides which side is favored. The next stage tested whether such profiles keep their effect outside the data and pipeline on which they were tuned.

Table 5 Identity-aware profile versus its baseline (YOLOv8n with ByteTrack at default settings) on VisDrone test-dev, historical protocol. Gain: paired bootstrap difference with 95% interval; for IDS it is the reduction.

Metric	Baseline	Identity	Gain [95% CI]
HOTA	28.430	31.218	+2.788 [0.747, 4.660]
MOTA	19.729	23.792	+4.063 [0.952, 6.886]
IDF1	32.724	37.870	+5.146 [2.028, 8.080]
IDS	1235	919	316 [175, 470]
FPS	36.528	46.024	–

C. Transfer and cross-pipeline studies

This stage applied the historical controllers outside their development setting: to a second dataset, and to two other detector–tracker pipelines.

UAVDT. The baseline is again YOLOv8n with ByteTrack at default settings (Sec. IV.A). Both frozen profiles were applied to UAVDT [3] without retuning (Table 6). The quality-oriented profile raised HOTA by 4.305 points $[2.740, 5.709]$ and the identity-aware profile by 2.845 points $[1.238, 4.348]$; their IDS reductions are 237 $[104, 377]$ and 250 $[110, 404]$.

Table 6 Historical UAVDT transfer without retuning. Baseline: YOLOv8n with ByteTrack at default settings.

Metric	Baseline	Quality	Identity
HOTA	24.085	28.390	26.930
MOTA	13.841	17.399	16.118
IDF1	27.887	34.453	32.014
IDS	558	321	308
FPS	65.015	58.353	61.462

U2MOT on VisDrone. The baseline is the published U2MOT tracker [30] with its released YOLOX-X detector, run at its own static setting. Seven VisDrone validation sequences served for development and 17 test-dev sequences for evaluation. The cue weights and transforms of the quality-oriented profile were kept, the tracker was not modified, and the action boundaries were set to the 33rd and 67th percentiles of the index on the U2MOT validation run, without using ground truth. Table 7 (top) gives the comparison: the controlled pipeline has 87 fewer IDS and 1230 fewer FP, while MOTA is almost unchanged and FN are higher.

SparseTrack on MOT17. The baseline is the published SparseTrack tracker [31] with a fixed NMS IoU threshold of 0.75, on the seven MOT17 `val_half` sequences (2652 frames) [11]. A minimal edge-driven rule switched only the NMS threshold, between 0.70 and 0.80; its threshold came from a leave-one-sequence-out study on the same sequences. Table 7 (bottom) gives the comparison: the adaptive rule reduces IDS from 136 to 123, with MOTA +0.0371 points $[-0.041, +0.182]$.

Table 7 Cross-pipeline studies. Top: U2MOT with its YOLOX-X detector at its static setting on VisDrone test-dev. Bottom: SparseTrack with a fixed NMS IoU threshold of 0.75 on MOT17 `val_half`.

Pipeline	Metric	Baseline	With control
U2MOT	MOTA	53.873	53.767
	IDF1	69.775	69.849
	IDS	1239	1152
	FP	41385	40155
	FN	63241	64801
SparseTrack	MOTA	76.8881	76.9252
	IDF1	81.4943	81.5299
	IDS	136	123

What this stage showed. The frozen profiles kept their direction on UAVDT without retuning, and with U2MOT and SparseTrack the control mainly reduced IDS, with small changes in MOTA. Together these studies show that the preferred profile depends on the objective (quality or identity) and on the tracker host. This motivated a modern stage with a stronger detector, explicit calibration data, and two tracker hosts.

D. Modern development of the frozen controller

The modern stage decided which cues enter the SCI, which operating points each state uses, and where the SCI thresholds lie.

Baseline. The modern baseline is the YOLO11m detector [23], in the public VisDrone checkpoint described in Sec. II.C.1, run at a fixed operating point of 1536 pixels and NMS IoU 0.70 with no extra score threshold. It feeds ByteTrack [6] during development and both ByteTrack and OATrack [22] in the final evaluation. A large fixed input is a common choice for small aerial objects when no adaptation is used.

Data and protocol. Eight sequences of the VisDrone2019-MOT training split (3282 frames) were declared as calibration data. They are disjoint from the seven evaluation sequences. The searches used the same four folds, each holding out two calibration sequences. Detector outputs were cached for every resolution and NMS value; a score threshold applied to the cache was checked to reproduce direct inference exactly. The steps are described below in the order in which they were carried out.

Resolution and NMS maps. Two exhaustive grid searches, archived as Stages 1 and 2, searched the state-to-resolution and state-to-NMS maps. At this point the action of a state contained only the input resolution and the NMS value.

Cue audit. The next search, archived as Stage 3A, asked which cues should form the SCI. It searched all 31 non-empty subsets of the five cues and selected {crowd}, which defines the crowd-selected SCI of Eq. (15). The other cues were kept as guard and auxiliary cues of the state logic (Sec. III.C).

Score threshold. A separate sweep then tested the score threshold. At 1088 pixels and NMS 0.45, a fixed score threshold of 0.40 raised pooled calibration HOTA by +0.881 over 0.01 and improved seven of the eight sequences. The score threshold was therefore added as the third action dimension.

Final joint search. The final search then selected the thresholds and the three-part actions together. It used 60 TPE trials (seed 43) and searched $\tau_m \in [0.10, 0.50]$, $\tau_h \in [\tau_m + 0.05, \min(\tau_m + 0.40, 0.90)]$, and for each state a resolution in $\{1088, 1280, 1536\}$, an NMS value in $\{0.45, 0.60, 0.70\}$, and a score threshold in $\{0.10, 0.25, 0.40\}$. The objective was the cross-validated (CV) mean change in HOTA against the baseline, with ByteTrack as host. A candidate was eligible only if it really adapted within the video rather than acting as one fixed operating point: it had to realize at least two distinct actions, each on at least 10% of calibration frames, and switch within sequences. The frozen policy is the eligible candidate whose three states use three different actions that together vary all three parts of the operating point (Table 2). On calibration data it used LOW, MEDIUM, and HIGH on 7.6%, 10.1%, and 82.3% of frames, with 0.607

state changes per 100 frames on average.

Comparison. Table 8 compares the frozen policy with the baseline on each calibration fold. All four folds are positive, with a CV mean of +2.447 HOTA. After these steps the thresholds, cue subset, and action map were frozen. The final comparison on the evaluation sequences is given in Sec. V.

Table 8 Frozen policy versus the modern baseline (YOLO11m at 1536 pixels and NMS 0.70, ByteTrack) on the four calibration folds: change in HOTA.

	Fold 1	Fold 2	Fold 3	Fold 4	Mean
ΔHOTA	+0.592	+4.164	+2.029	+3.004	+2.447

V. Results

The experiments test whether replacing the fixed operating point by the frozen AC-MOT controller improves tracking with the same detector and tracker, on VisDrone and, without retuning, on UAVDT.

All experiments use the YOLO11m detector of Sec. II.C.1. It runs in half precision, and detections of the classes pedestrian, car, van, truck, and bus are passed on. ByteTrack uses the Ultralytics implementation with its default parameters: high threshold 0.25, low threshold 0.10, new-track threshold 0.25, buffer 30 frames, and match threshold 0.80. OATrack is a re-implementation of the published method [22] in its main configuration, with its published tracker-side confidence gate of 0.40. This gate belongs to the tracker and is never changed by AC-MOT. For each host, two systems are compared. The baseline runs the detector at the fixed operating point of 1536 pixels and NMS IoU 0.70, with a minimal score threshold of 0.01 (Sec. III.F). AC-MOT runs the frozen controller of Sec. III on the same detector. The two systems differ only in the detector operating point. Uncertainty is estimated with the paired bootstrap of Sec. II.B, with 5000 resamples (seed 42). Latency is measured on one Tesla T4 GPU over the first 200 frames of sequence `uav0000086_00000_v`, excluding five warm-up frames. For every frame, the time spent in the controller (image statistics, scene analysis, and state update) is recorded separately from the detector and tracker time.

A. Frozen VisDrone evaluation

The VisDrone evaluation uses seven sequences of the VisDrone2019-MOT validation split. They contain 71,830 ground-truth boxes. The cue subset, thresholds, and action map of the frozen controller were selected on the separate calibration sequences only (Sec. IV.D). Results are scored with a Python port of the VisDrone2019-MOT Task 4b evaluation. Ignored regions are removed, the five classes above are evaluated separately, CLEAR-MOT and identity measures use IoU 0.5, and counts are summed over classes and sequences. HOTA is added with the same class-aware scheme using TrackEval. The bootstrap draws seven sequences with replacement.

Table 9 lists the aggregate results on the seven VisDrone sequences, and Fig. 2 shows HOTA, MOTA, and IDF1.

ByteTrack host. AC-MOT raises HOTA, MOTA, and IDF1 and lowers IDS and FP, while FN rise from 24121 to

Table 9 Frozen VisDrone evaluation on seven VisDrone2019-MOT sequences (71,830 ground-truth boxes). Baseline: fixed operating point (1536, 0.70).

	ByteTrack		OATrack	
Metric	Baseline	AC-MOT	Baseline	AC-MOT
HOTA	41.586	44.523	46.770	48.095
MOTA	9.876	27.871	28.165	34.483
IDF1	48.569	53.929	57.107	59.364
IDS	1341	1016	773	604
FP	39274	24892	25394	21400
FN	24121	25902	25432	25057
Precision	54.849	64.852	64.628	68.609
Recall	66.419	63.940	64.594	65.116

25902 and recall falls from 66.419 to 63.940. The detector output becomes more selective: clutter and identity-confusing boxes are removed at a small cost in recall. The very low baseline MOTA shows how many FP the fixed operating point passes to this tracker; because MOTA counts every FP, the removal of more than a third of them is the main source of the MOTA gain.

OATrack host. OATrack starts from a stronger baseline, and its gains are smaller, but every metric of Table 9 moves in the direction of improvement, including FN (25432 to 25057) and recall (64.594 to 65.116).

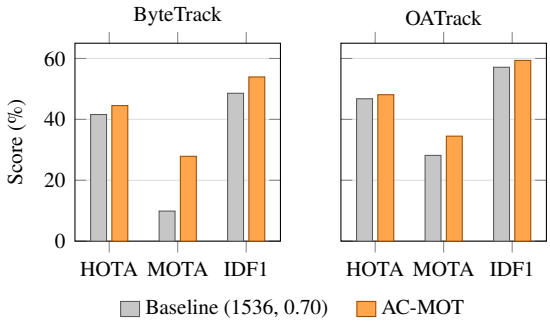


Fig. 2 HOTA, MOTA, and IDF1 on the seven VisDrone sequences for both tracker hosts (values in Table 9).

Paired bootstrap. In the VisDrone columns of Table 10, the ByteTrack intervals of HOTA, MOTA, IDF1, precision, IDS, and FP lie entirely on the side of improvement, while FN and recall move in the opposite direction. With OATrack, HOTA, MOTA, IDF1, and precision rise and IDS and FP fall, all with intervals on one side of zero. Because whole sequences are resampled, the intervals reflect the variation between videos, not only between frames.

Controller behavior. The controller uses all three states with both hosts. With ByteTrack, LOW, MEDIUM, and HIGH cover 7.4%, 30.1%, and 62.5% of the frames; with OATrack they cover 7.4%, 33.3%, and 59.3%. For most of the video, the detector therefore runs with the HIGH action, the smallest input and a score threshold of 0.40. The two hosts diverge through their different tracked boxes. State changes are rare, between 0.10 and 0.54 per 100 frames depending on the sequence, so the detector input stays stable over long stretches of video.

Table 10 Paired sequence-level bootstrap: AC-MOT minus baseline, 5000 resamples, seed 42, on VisDrone (7 sequences) and UAVDT (20 sequences). HOTA, MOTA, IDF1, precision, and recall in points; IDS, FP, and FN in counts. On UAVDT the bootstrap covers the count-based metrics only.

		VisDrone		UAVDT	
Host	Metric	Difference	95% interval	Difference	95% interval
ByteTrack	HOTA	+2.937	[+2.356, +3.375]	–	
	MOTA	+17.995	[+13.541, +23.282]	+17.601	[+11.134, +26.767]
	IDF1	+5.360	[+4.490, +6.178]	+3.367	[+1.676, +5.013]
	IDS	−325	[−498, −164]	−115	[−211.025, −26.975]
	FP	−14382	[−19176, −9620]	−67539	[−104215.350, −37352.950]
	FN	+1781	[+288, +3594.075]	+7652	[+1542.925, +18359.400]
	Precision	+10.003	[+8.712, +11.895]	–	
	Recall	−2.479	[−4.507, −0.469]	–	
OATrack	HOTA	+1.325	[+0.200, +2.264]	–	
	MOTA	+6.318	[+4.741, +7.879]	+3.267	[+0.607, +5.881]
	IDF1	+2.258	[+0.548, +3.474]	+0.831	[+0.125, +1.271]
	IDS	−169	[−292, −42]	−31	[−101, +39]
	FP	−3994	[−6190, −2494.900]	−14268	[−33958.075, −2188.950]
	FN	−375	[−1462.575, +670]	+3162	[−211.100, +8959.050]
	Precision	+3.981	[+2.957, +5.511]	–	
	Recall	+0.522	[−0.869, +2.111]	–	

B. Frozen UAVDT transfer

The UAVDT transfer tests whether the frozen controller still helps on a dataset with other scenes, object types, and recording conditions, with none of its values changed. It uses 20 test sequences with 16,592 frames, scored with one vehicle class. Detector outputs were produced at the three frozen operating points (49,776 detector forward passes), with a single frozen run per host. The paired bootstrap draws 20 sequences with replacement from the stored per-sequence counts and therefore covers the count-based metrics MOTA, IDF1, IDS, FP, and FN.

Table 11 Frozen UAVDT transfer without retuning (20 sequences, 16,592 frames, one vehicle class). Baseline: fixed operating point (1536, 0.70).

	ByteTrack		OATrack	
Metric	Baseline	AC-MOT	Baseline	AC-MOT
HOTA	45.923	47.004	47.728	47.640
MOTA	−1.573	16.028	19.607	22.873
IDF1	58.400	61.767	62.101	62.931
IDS	601	486	517	486
FP	279616	212077	197937	183669
FN	66050	73702	75612	78774
Precision	49.571	55.751	57.270	58.800
Recall	80.625	78.381	77.820	76.893

Table 11 and Fig. 3 report the transfer to UAVDT without retuning. With ByteTrack, HOTA rises by 1.081 points, MOTA by 17.601, and IDF1 by 3.367; IDS fall by 115 and FP by 67539, while FN rise by 7652. With OATrack, MOTA rises by 3.267 and IDF1 by 0.831; IDS fall by 31 and FP by 14268, while HOTA is almost unchanged (47.728 and

47.640). With both hosts the transfer gives fewer FP and IDS and higher precision, with slightly lower recall, the same pattern as with ByteTrack on VisDrone: the controller changes the detector output in the same way on a dataset that was not used to build it.

The UAVDT columns of Table 10 give the paired bootstrap differences. With ByteTrack, the intervals of MOTA, IDF1, IDS, and FP lie entirely on the side of improvement, and FN moves in the opposite direction. With OATrack, MOTA and IDF1 rise and FP fall, with intervals on one side of zero, while the intervals of IDS and FN include zero.

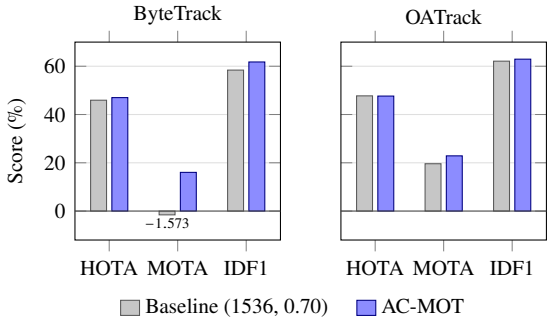


Fig. 3 HOTA, MOTA, and IDF1 of the frozen UAVDT transfer for both tracker hosts (values in Table 11). The ByteTrack baseline MOTA is negative (−1.573); the solid line marks zero.

VI. Discussion

The operating point is part of the MOT system. The same frozen detector and the same trackers give very different MOT results at different operating points. Moving away from the fixed 1536/0.70 point raises MOTA by 17.995 points with ByteTrack and by 6.318 points with OATrack on VisDrone. The detector operating point is therefore a first-order variable of aerial MOT and deserves the same attention as the association method.

A more selective detection stream. On both datasets, FP and IDS drop sharply and precision rises, while FN rise a little. MOTA reacts most strongly because it counts every FP; HOTA and IDF1 rise by smaller amounts.

Tracker dependence. ByteTrack gains more than OATrack. ByteTrack starts from a weaker baseline and uses low-score boxes in its second association step, so a likely reason is that removing clutter helps it directly. OATrack already weights detections by quality inside its association, and it gains mainly in MOTA, IDF1, IDS, and FP. Results are therefore reported per host.

Development path. The historical profiles showed that the preferred operating point depends on the goal and on the tracker host; the modern stage turned these observations into a small controller with one crowd index, a few guard cues, three states, and three actions.

Deployment. AC-MOT needs no training and no access to detector or tracker internals; it changes three arguments of the detector call and analyzes a quarter-scale image every ten frames. On a Tesla T4 the controller takes 4.26 ms per frame with ByteTrack and 4.20 ms with OATrack, which is small next to detector inference, so it is suitable for real-time

use. The same interface can host other action maps, for example maps chosen for a latency budget, without changes to the detector or the tracker.

VII. Conclusion

This paper presented AC-MOT, a training-free method that adjusts the detector operating point online to improve MOT in UAV videos. Instead of using fixed detector settings, AC-MOT changes the confidence threshold, the NMS IoU threshold, and the input resolution as the scene changes. It uses only the current frame and previous tracking results, and it does not change or retrain the detector or the tracker. The SCI, which is based on object density, together with a few other scene cues and a three-state controller with recovery, hysteresis, and dwell rules, lets the settings follow the scene without frequent changes.

The controller was developed through optimization experiments, and its final values were selected by cross-validation on separate calibration sequences and then kept fixed.

On seven VisDrone2019-MOT sequences, AC-MOT improved HOTA by 2.937 points with ByteTrack and 1.325 points with OATrack, and MOTA by 17.995 and 6.318 points, respectively; paired bootstrap analysis gave confidence intervals for these differences. On UAVDT, the same controller improved MOTA by 17.601 points with ByteTrack and 3.267 points with OATrack without retuning.

Overall, the results show that the detector operating point should be treated as part of the tracking system rather than as a fixed setting. Future work will test the controller with more detectors, trackers, and aerial video datasets.

References

[1] Wu, X., Li, W., Hong, D., Tao, R., and Du, Q., "Deep Learning for Unmanned Aerial Vehicle-Based Object Detection and Tracking: A survey," *IEEE Geoscience and Remote Sensing Magazine*, Vol. 10, No. 1, 2022, pp. 91–124. doi:10.1109/mgrs.2021.3115137.

[2] Zhu, P., Wen, L., Du, D., Bian, X., Fan, H., Hu, Q., and Ling, H., "Detection and Tracking Meet Drones Challenge," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, Vol. 44, No. 11, 2022, pp. 7380–7399. doi:10.1109/tpami.2021.3119563.

[3] Du, D., Qi, Y., Yu, H., Yang, Y., Duan, K., Li, G., Zhang, W., Huang, Q., and Tian, Q., "The Unmanned Aerial Vehicle Benchmark: Object Detection and Tracking," *Computer Vision – ECCV 2018*, Springer International Publishing, Cham, 2018, pp. 375–391. doi:10.1007/978-3-030-01249-6_23.

[4] Cheng, G., Yuan, X., Yao, X., Yan, K., Zeng, Q., Xie, X., and Han, J., "Towards Large-Scale Small Object Detection: Survey and Benchmarks," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2023, pp. 1–20. doi:10.1109/tpami.2023.3290594.

[5] Bewley, A., Ge, Z., Ott, L., Ramos, F., and Upcroft, B., "Simple online and realtime tracking," *2016 IEEE International Conference on Image Processing (ICIP)*, IEEE, Phoenix, AZ, USA, 2016, pp. 3464–3468. doi:10.1109/icip.2016.7533003.

[6] Zhang, Y., Sun, P., Jiang, Y., Yu, D., Weng, F., Yuan, Z., Luo, P., Liu, W., and Wang, X., "ByteTrack: Multi-object Tracking by Associating Every Detection Box," *Computer Vision – ECCV 2022*, Springer Nature Switzerland, Cham, 2022, pp. 1–21. doi:10.1007/978-3-031-20047-2_1.

[7] Chin, T.-W., Ding, R., and Marculescu, D., "AdaScale: Towards Real-time Video Object Detection using Adaptive Scaling," *Proceedings of Machine Learning and Systems*, Vol. 1, 2019, pp. 431–441.

[8] Xu, R., lin Zhang, C., Wang, P., Lee, J., Mitra, S., Chaterji, S., Li, Y., and Bagchi, S., "ApproxDet: Content and Contention-Aware Approximate Object Detection for Mobiles," *Proceedings of the 18th Conference on Embedded Networked Sensor Systems (SenSys)*, ACM, 2020, pp. 449–462. doi:10.1145/3384419.3431159.

[9] Ma, L. V., Hussain, M. I., Park, J., Kim, J., and Jeon, M., "Adaptive Confidence Threshold for ByteTrack in Multi-Object Tracking," *2023 12th International Conference on Control, Automation and Information Sciences (ICCAIS)*, IEEE, 2023, pp. 370–374. doi:10.1109/iccais59597.2023.10382403.

[10] Shim, K., Ko, K., Hwang, J., and Kim, C., "AdapTrack: Adaptive Thresholding-Based Matching for Multi-Object Tracking," *2024 IEEE International Conference on Image Processing (ICIP)*, IEEE, 2024, pp. 2222–2228. doi:10.1109/icip51287.2024.10648139.

[11] Dendorfer, P., Ošep, A., Milan, A., Schindler, K., Cremers, D., Reid, I., Roth, S., and Leal-Taixé, L., "MOTChallenge: A Benchmark for Single-Camera Multiple Target Tracking," *International Journal of Computer Vision*, Vol. 129, No. 4, 2021, pp. 845–881. doi:10.1007/s11263-020-01393-0.

[12] Luiten, J., Ošep, A., Dendorfer, P., Torr, P., Geiger, A., Leal-Taixé, L., and Leibe, B., "HOTA: A Higher Order Metric for Evaluating Multi-object Tracking," *International Journal of Computer Vision*, Vol. 129, No. 2, 2021, pp. 548–578. doi:10.1007/s11263-020-01375-2.

[13] Efron, B., and Tibshirani, R. J., *An Introduction to the Bootstrap*, Springer US, New York, 1993. doi:10.1007/978-1-4899-4541-9.

[14] Du, Y., Zhao, Z., Song, Y., Zhao, Y., Su, F., Gong, T., and Meng, H., "StrongSORT: Make DeepSORT Great Again," *IEEE Transactions on Multimedia*, Vol. 25, 2023, pp. 8725–8737. doi:10.1109/tmm.2023.3240881.

[15] Meinhardt, T., Kirillov, A., Leal-Taixe, L., and Feichtenhofer, C., "TrackFormer: Multi-Object Tracking With Transformers," *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 8844–8854.

[16] Liu, S., Li, X., Lu, H., and He, Y., "Multi-Object Tracking Meets Moving UAV," *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, IEEE, 2022, pp. 8866–8875. doi:10.1109/cvpr52688.2022.00867.

[17] Miller, N., and Rogers, J., "Simultaneous Tracking of Multiple Ground Targets from a Multirotor Unmanned Aerial Vehicle," *Journal of Aerospace Information Systems*, Vol. 12, No. 3, 2015, pp. 345–364. doi:10.2514/1.i010345.

[18] Desai, A., and Lee, D.-J., "Efficient Feature Descriptor for Unmanned Aerial Vehicle Ground Moving Object Tracking," *Journal of Aerospace Information Systems*, Vol. 14, No. 6, 2017, pp. 345–350. doi:10.2514/1.i010503.

[19] Lu, Y.-R., and Sun, D., "Development of Real-Time Unmanned Aerial Vehicle Urban Object Detection System with Federated Learning," *Journal of Aerospace Information Systems*, Vol. 21, No. 7, 2024, pp. 547–553. doi:10.2514/1.i011378.

[20] Choi, A. J., Park, J., and Han, J.-H., "Automated Aerial Docking System Using Onboard Vision-Based Deep Learning," *Journal of Aerospace Information Systems*, Vol. 19, No. 6, 2022, pp. 421–436. doi:10.2514/1.i011053.

[21] Dai, P., Chen, L., Lian, Y., and Xie, L., "LSO-YOLO: a lightweight real-time object detection network for UAV small-object detection," *Journal of Real-Time Image Processing*, Vol. 23, No. 5, 2026, p. 176. doi:10.1007/s11554-026-01977-y.

[22] Qi, L., Wang, L., and Jiang, S., "OATrack: A Quality-Gated Progressive Association Framework with a YOLO Detection Cache for UAV Small-Object Multi-Object Tracking," *Sensors*, Vol. 26, No. 16, 2026, p. 5222. doi:10.3390/s26165222.

[23] Jocher, G., and Qiu, J., "Ultralytics YOLO11," Software package, Ver. 11.0.0, Ultralytics, `https://github.com/ultralytics/ultralytics`, 2024.

[24] Han, Y., Huang, G., Song, S., Yang, L., Wang, H., and Wang, Y., "Dynamic Neural Networks: A Survey," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, Vol. 44, No. 11, 2022, pp. 7436–7456. doi:10.1109/tpami.2021.3117837.

[25] Yang, L., Han, Y., Chen, X., Song, S., Dai, J., and Huang, G., "Resolution Adaptive Networks for Efficient Inference," *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 2369–2378.

[26] Jiang, J., Ananthanarayanan, G., Bodik, P., Sen, S., and Stoica, I., "Chameleon: Scalable Adaptation of Video Analytics," *Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication*, ACM, New York, NY, USA, 2018, pp. 253–266. doi:10.1145/3230543.3230574.

[27] Akiba, T., Sano, S., Yanase, T., Ohta, T., and Koyama, M., "Optuna: A Next-generation Hyperparameter Optimization Framework," *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ACM, Anchorage, AK, USA, 2019, pp. 2623–2631. doi:10.1145/3292500.3330701.

[28] Lin, T.-Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P., and Zitnick, C. L., "Microsoft COCO: Common Objects in Context," *Computer Vision – ECCV 2014*, Springer International Publishing, Cham, 2014, pp. 740–755. doi:10.1007/978-3-319-10602-1_48.

[29] Jocher, G., Chaurasia, A., and Qiu, J., "Ultralytics YOLOv8," Software package, Ver. 8.0.0, Ultralytics, `https://github.com/ultralytics/ultralytics`, 2023.

[30] Liu, K., Jin, S., Fu, Z., Chen, Z., Jiang, R., and Ye, J., "Uncertainty-aware Unsupervised Multi-Object Tracking," *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023, pp. 9996–10005.

[31] Liu, Z., Wang, X., Wang, C., Liu, W., and Bai, X., "SparseTrack: Multi-Object Tracking by Performing Scene Decomposition Based on Pseudo-Depth," *IEEE Transactions on Circuits and Systems for Video Technology*, Vol. 35, No. 5, 2025, pp. 4870–4882. doi:10.1109/TCSVT.2024.3524670.